

Design Pattern

Builder

Adam HAMOUC

May 2026

1. Intention

Construire des objets complexes **etape par etape**, en separant la construction de la representation finale. Permet de produire des variantes du meme objet avec le meme code de construction.

Alternatives qu'il remplace :

- Constructeur geant avec des dizaines de parametres optionnels.
- Explosion de sous-classes pour couvrir toutes les variantes.

2. Probleme resolu

```
// Constructeur geant : illisible, parametres optionnels ambigus
VkPipeline p = createPipeline(shader, rp, true, false,
                             VK_CULL_MODE_BACK_BIT, 0, nullptr);
// Que font "true" et "false" ? Impossible a lire sans doc.
```

3. Structure

- **IBuilder** : interface declarant toutes les etapes de construction. Chaque methode retourne ***this** pour permettre le chainage fluent.
- **Builders concrets** : heritent de IBuilder. Chacun produit une variante du produit final en implementant les etapes differemment.
- **Director** : connait uniquement IBuilder. Definit des sequences completes predefinies et peut switcher de builder sans modifier son code.
- **Produit** : le resultat final. Les produits n'heritent pas forcement de la meme classe.
- **Reset** : build() retourne le produit et reinitialise le builder pour la prochaine construction.

4. Exemple — Pipeline Vulkan

```
// Interface IBuilder
class IPipelineBuilder {
public:
    virtual IPipelineBuilder& setShaders(VkShaderModule v, VkShaderModule f) = 0;
    virtual IPipelineBuilder& setRenderPass(VkRenderPass rp) = 0;
    virtual IPipelineBuilder& enableDepthTest(bool e) = 0;
    virtual IPipelineBuilder& enableBlending(bool e) = 0;
    virtual VkPipeline build() = 0;
    virtual ~IPipelineBuilder() = default;
};

// Builder concret : pipeline opaque
class OpaquePipelineBuilder : public IPipelineBuilder {
public:
    IPipelineBuilder& setShaders(VkShaderModule v, VkShaderModule f) override
    { cfg.vert = v; cfg.frag = f; return *this; }
    IPipelineBuilder& enableBlending(bool e) override
    { cfg.blending = e; return *this; }
    // ... autres etapes
```

```

    VkPipeline build() override {
        VkPipeline result = createVkPipeline(cfg);
        cfg = {}; // reset
        return result;
    }
private:
    PipelineConfig cfg;
};

// Builder concret : shadow pipeline (depth only, frag ignore)
class ShadowPipelineBuilder : public IPipelineBuilder {
    IPipelineBuilder& setShaders(VkShaderModule v, VkShaderModule) override
    { cfg.vert = v; return *this; } // frag shader ignore
    IPipelineBuilder& enableBlending(bool) override
    { return *this; } // blending ignore
    // ...
};

// Director : sequences predefinies, ne connait que IBuilder
class PipelineDirector {
public:
    PipelineDirector(IPipelineBuilder& b) : builder(b) {}
    VkPipeline buildOpaque(VkShaderModule v, VkShaderModule f, VkRenderPass rp) {
        return builder.setShaders(v, f).setRenderPass(rp)
            .enableDepthTest(true).enableBlending(false).build();
    }
    VkPipeline buildTransparent(VkShaderModule v, VkShaderModule f, VkRenderPass rp) {
        return builder.setShaders(v, f).setRenderPass(rp)
            .enableDepthTest(true).enableBlending(true).build();
    }
private:
    IPipelineBuilder& builder;
};

// Usage : meme director, builders differents
OpaquePipelineBuilder opaqueBuilder;
ShadowPipelineBuilder shadowBuilder;
PipelineDirector director(opaqueBuilder);
VkPipeline gbuffer = director.buildOpaque(vert, frag, renderPass);
director = PipelineDirector(shadowBuilder); // swap de builder
VkPipeline shadow = director.buildOpaque(depthVert, {}, shadowPass);

```

Variante fluente sans Director : si on n'a pas besoin de sequences predefinies ni de plusieurs builders, on peut se passer de l'interface et du Director. Le chainage reste disponible : `PipelineBuilder{}.setShaders(v,f).enableDepthTest(true).build();`

5. Avantages / Quand utiliser

Avantages :

- Construction lisible et explicite meme pour des objets tres complexes.
- Le Director produit plusieurs variantes avec le meme code de construction.
- Chaque etape est testable independamment.

Utiliser si :

- Objet avec beaucoup de parametres optionnels (`VkPipeline`, `RenderPass`, `Material`).
- Plusieurs variantes du meme objet a construire (opaque, transparent, shadow).

Eviter si : objet simple avec peu de parametres — un constructeur classique suffit.